Serverless Architectures

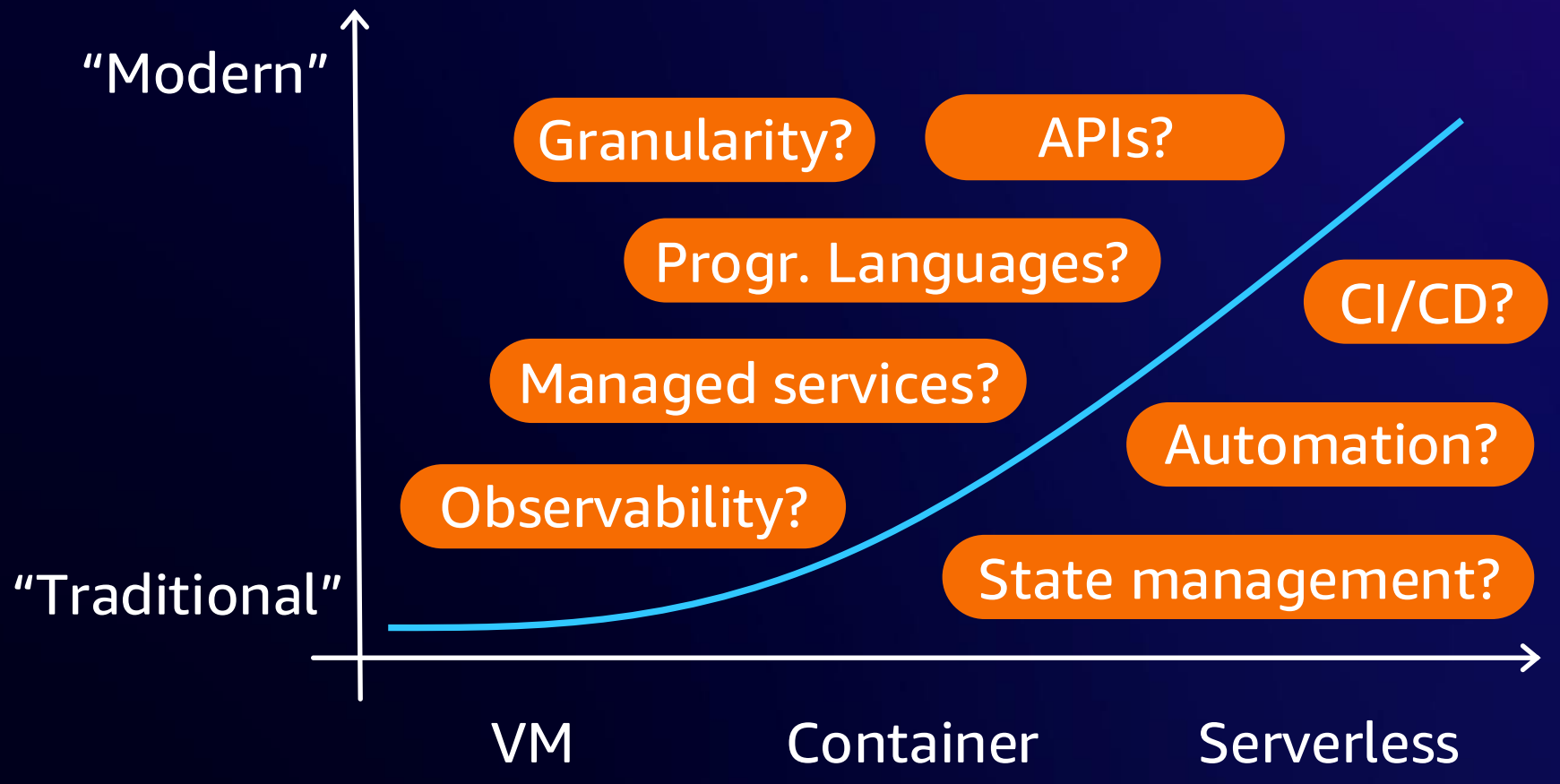
OK, NO MORE SERVERS – NOW WHAT?

Gregor Hohpe

Sr. Principal Evangelist
AWS

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How modern is your cloud application?



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Serverless – Not just a run-time

COMPUTE

AWS Lambda

AWS Fargate

DATA STORES

Amazon S3

Amazon Aurora Serverless

Amazon DynamoDB

INTEGRATION

Amazon EventBridge

Amazon API Gateway

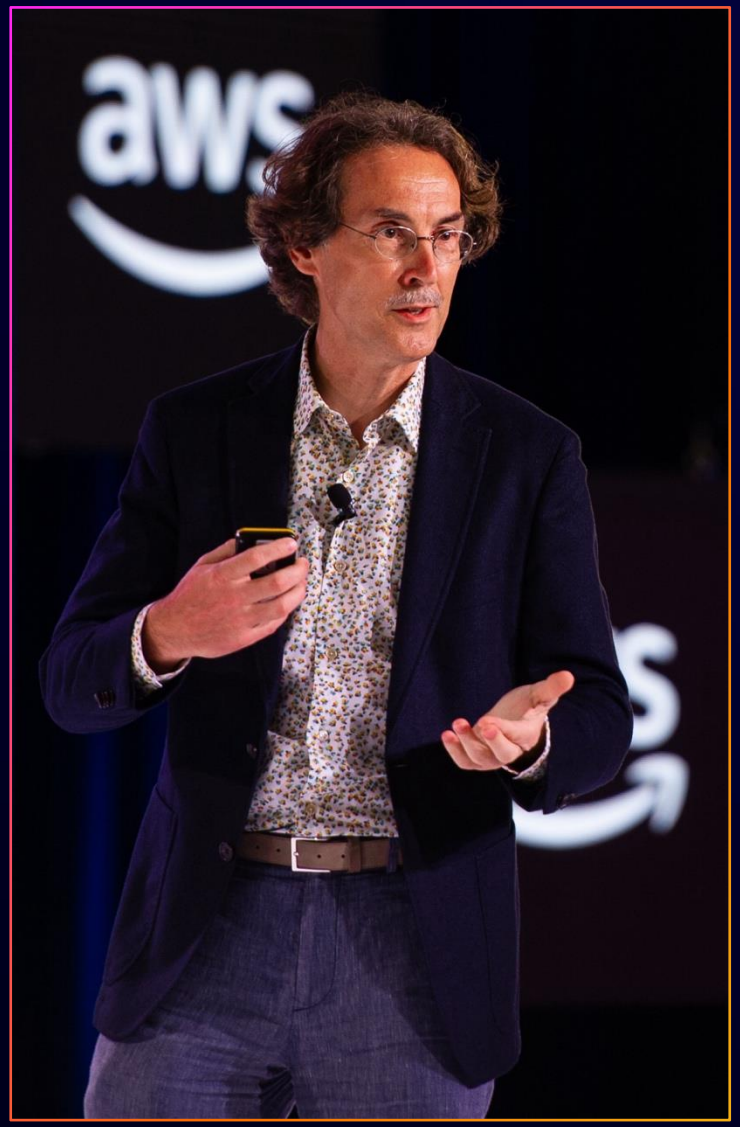
Amazon SQS

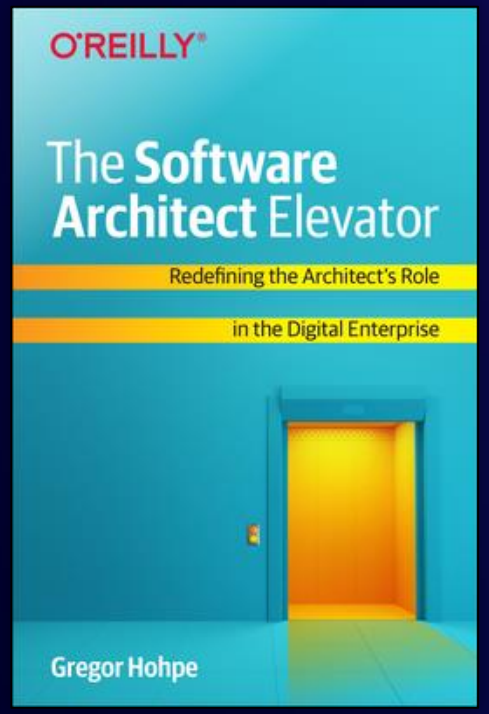
Amazon SNS

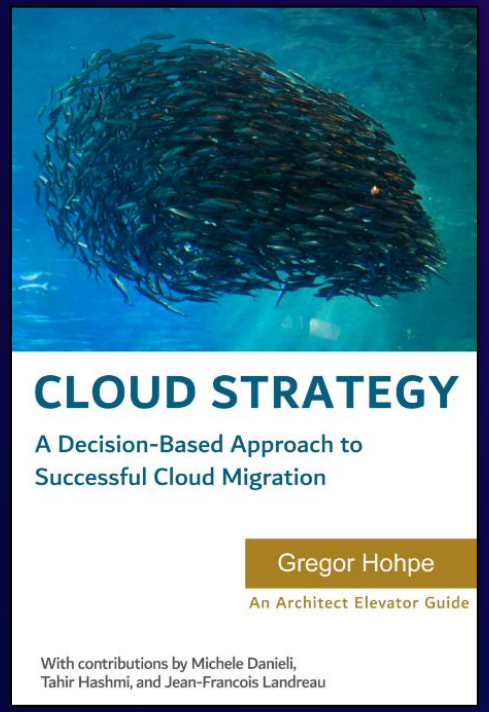
AWS Step Functions

AWS AppSync

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.







Sr. Principal Evangelist, Serverless

ArchitectElevator.com

www.linkedin.com/in/ghohpe/

Gregor Hohpe

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Serverless Architectures: Under the Lambda covers

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

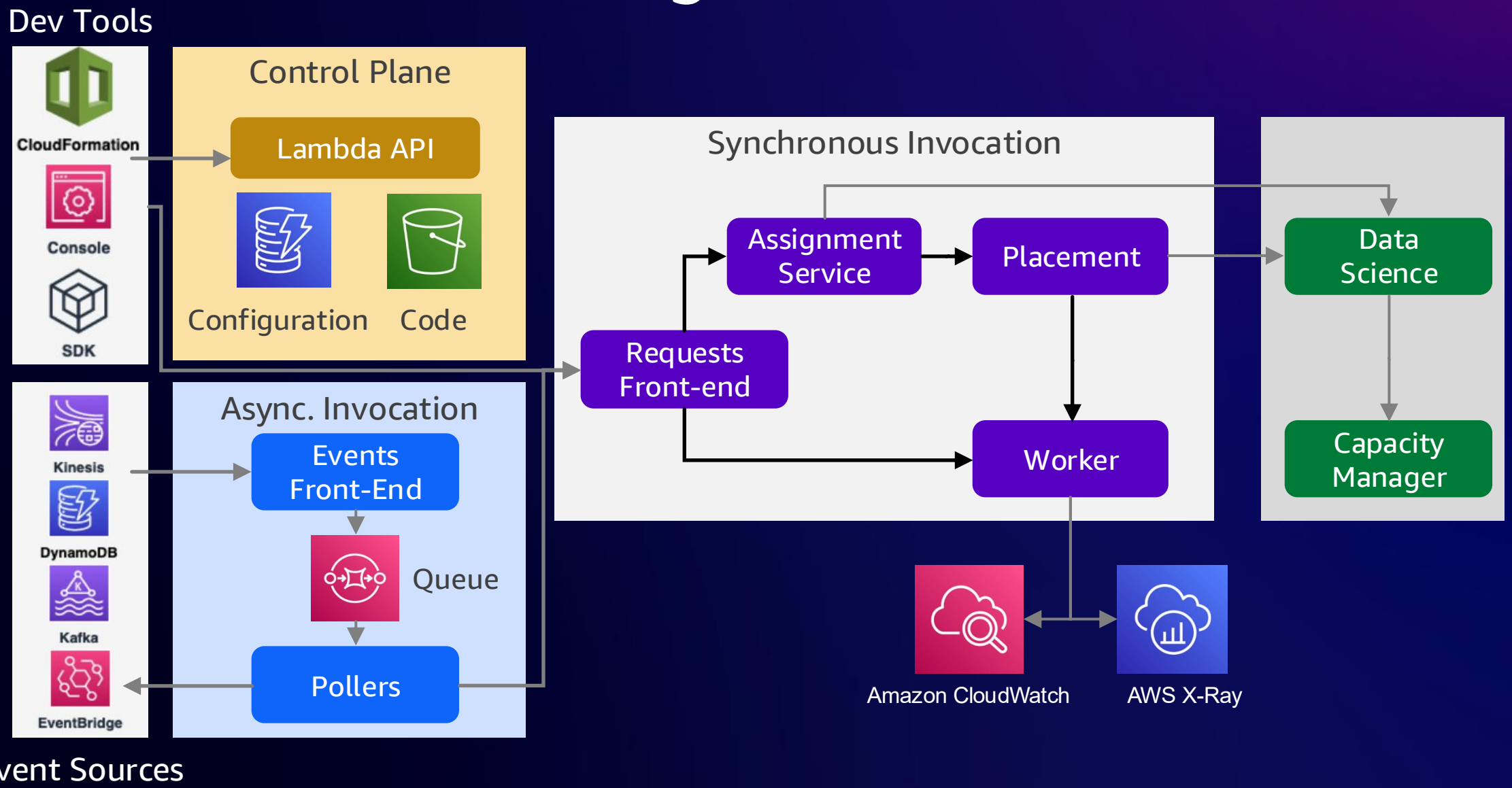
AWS Lambda – Launched Nov 2014

- Provide your code or image; we run it as an event when things happen
- Pay by the millisecond, up to 15mins
- Package as .zip file (250 MB) or container image (10 GB)
- Java, Go, Node.js, .NET, Python, and Ruby, or bring-your own
- No provisioning or managing servers
- Scale up in milliseconds in response to traffic

Lambda is the compression algorithm for experience

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lambda Architecture – High-level View



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Our experience: Implementation considerations

- Placement (Assignment Service)
  - Fault tolerant, leader-follower model
- Worker manager
- Isolation levels
  - Firecracker micro-VMs
  - Tenant de-correlation
- Initialization / cold starts
  - SnapStart
  - Chunks, read-ahead

https://www.infoq.com/news/2023/10/aws-lambda-under-the-hood/

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Serverless Architectures: Of boxes and lines

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

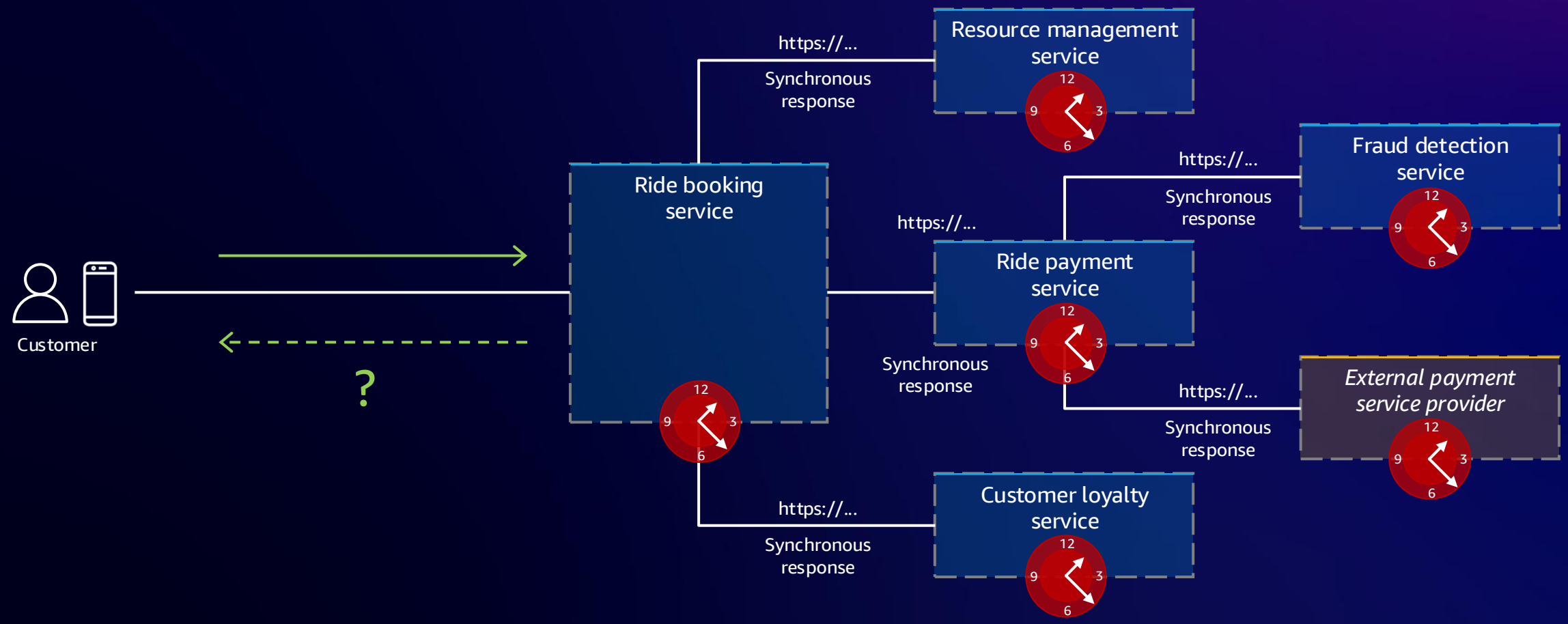
“

Serverless applications are distributed and fine-grained.

Distribute responsibly!

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Friends don’t let friends use synchronous integration



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

• Event-driven architectures

- Lambda has synchronous and asynchronous invocation models
- Push models, pull models, streaming
- Serverless integration provides orchestration, event routing, and message queues

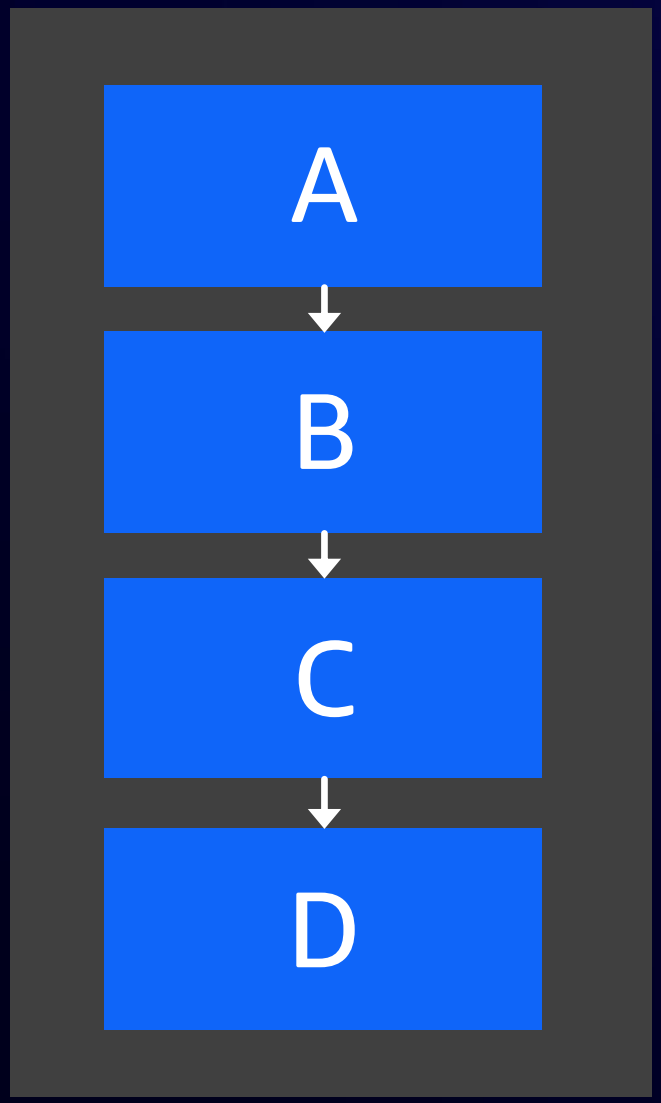
Distributed, event-driven architecture

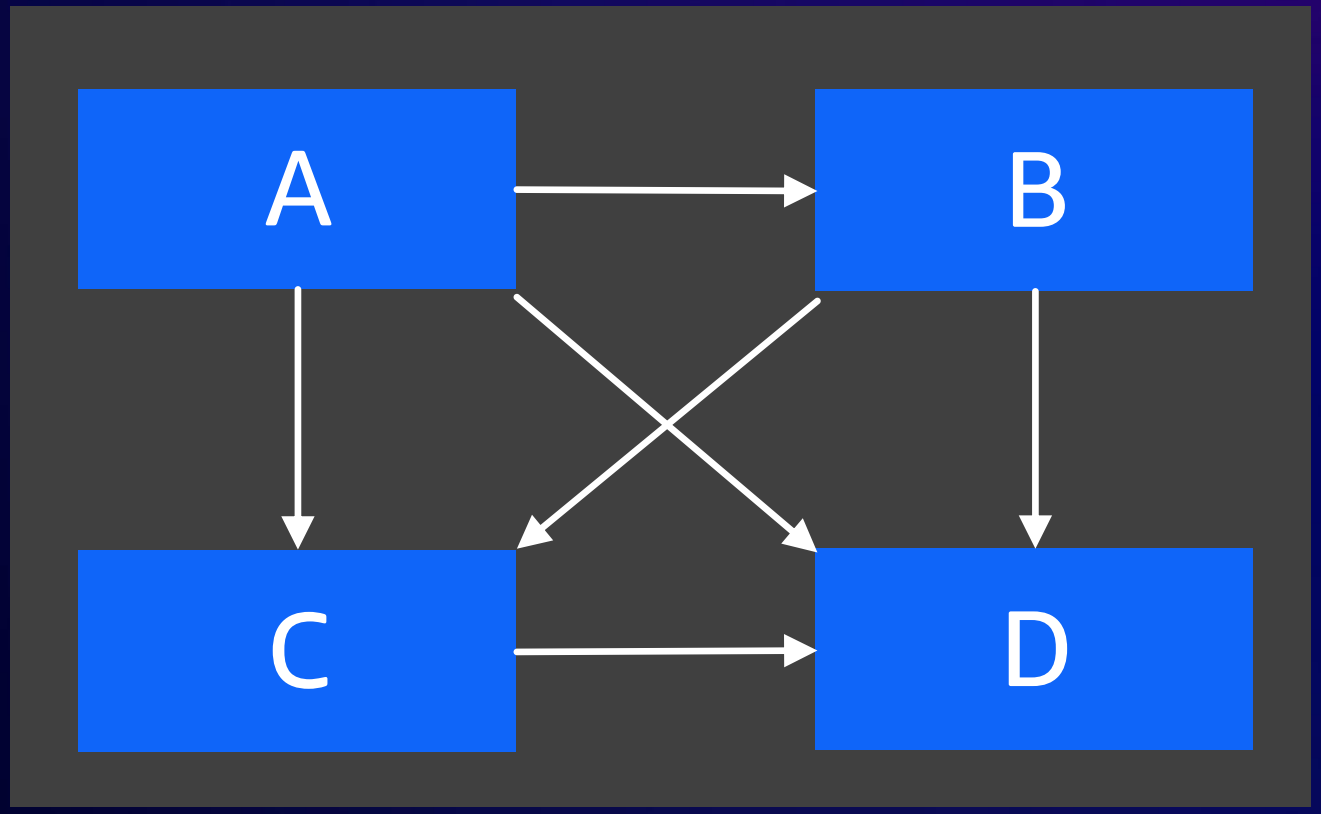
Reduced Dependencies	Independent Scalability	Failure Isolation	Evolvable Architecture

aws

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Behold the lines!





© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Connecting two systems. How hard can it be?

Data or control flow?

Pub-sub or point-to-point?

Interaction model?

Sync/async?

Polling?

Data format?

Distributed?

Conversation State?

Systems or instances?

Error handling?

Idempotency?

Partial failures?

Retries?

Back-off?

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

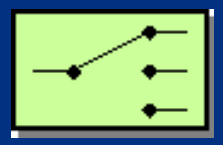
Control flow vs. data flow

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

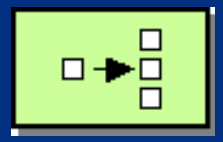
Enterprise Integration Patterns

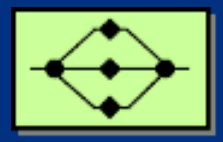
Data (Message) Flow

Message Routing

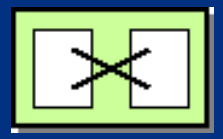

Content-based Router

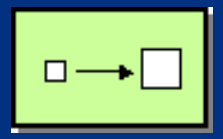
Message Filter

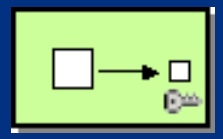

Splitter


Scatter Gather

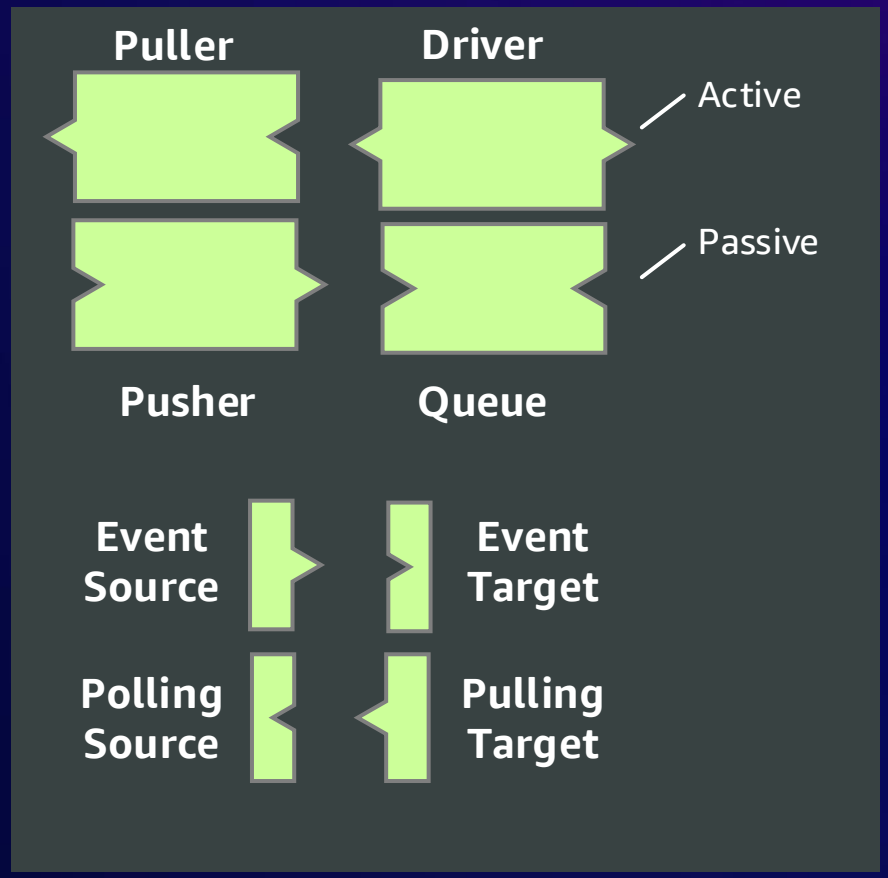
Message Transformation


Message Translator


Content Enricher


Claim Check

Control Flow

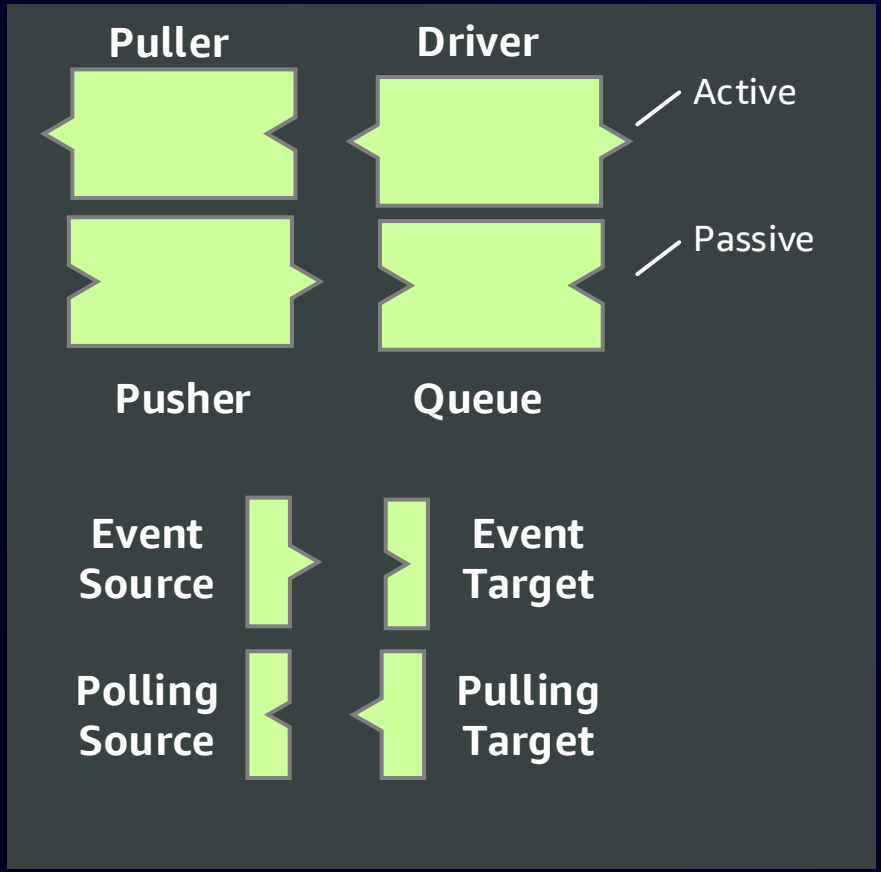


Source: EnterpriseIntegrationPatterns.com

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

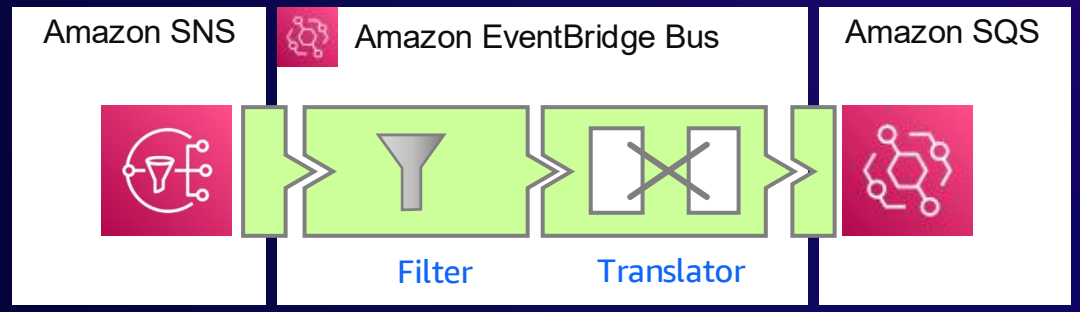
Visualizing control flow: EventBridge Bus

Control Flow

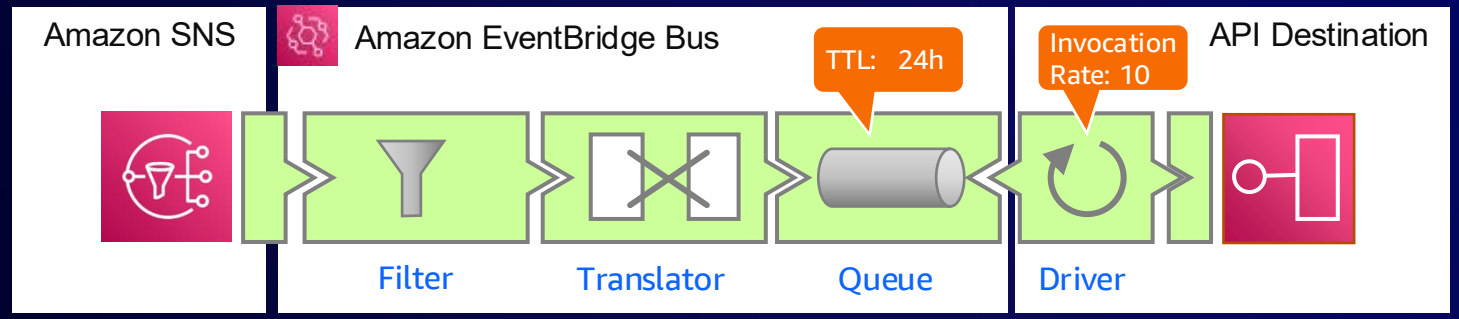


Source: EnterpriseIntegrationPatterns.com

EventBridge with Event Target



EventBridge with Rate Limited Target

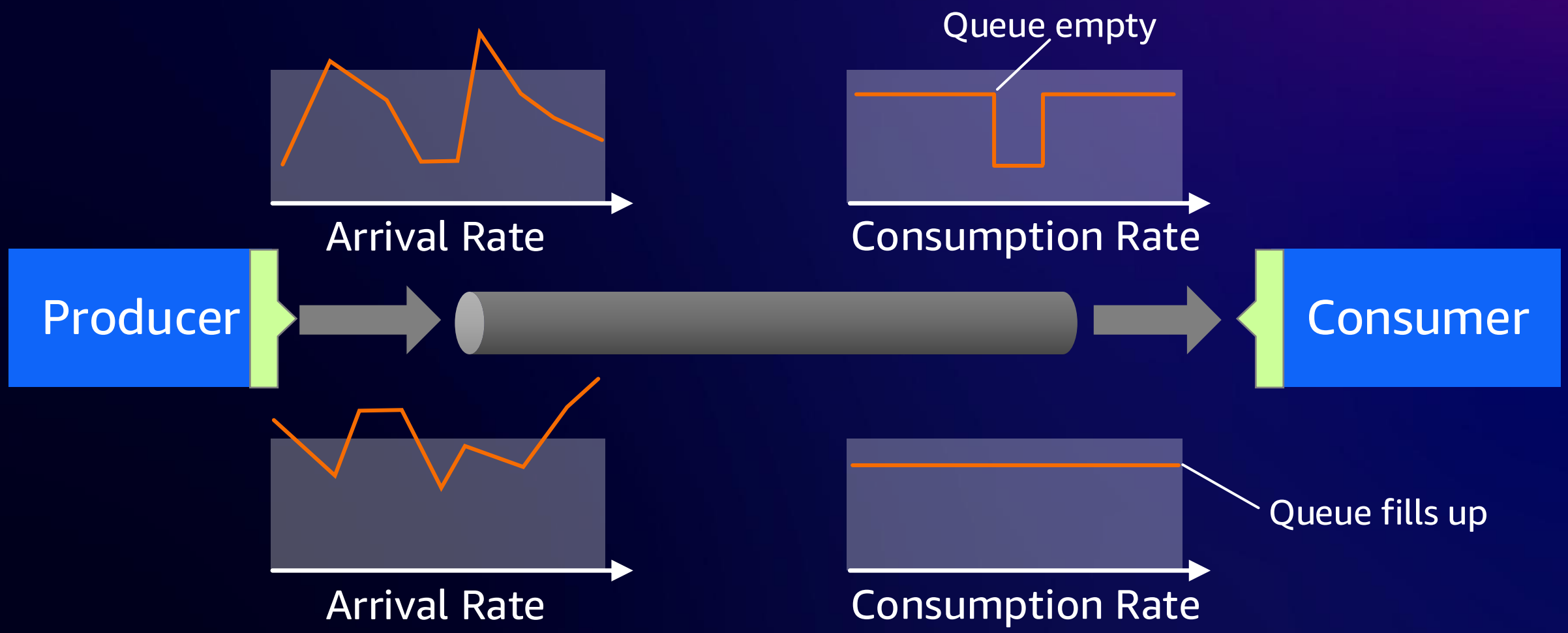


© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Control flow is essential to understanding dynamic system behavior such as latency, scaling, batching, time-outs

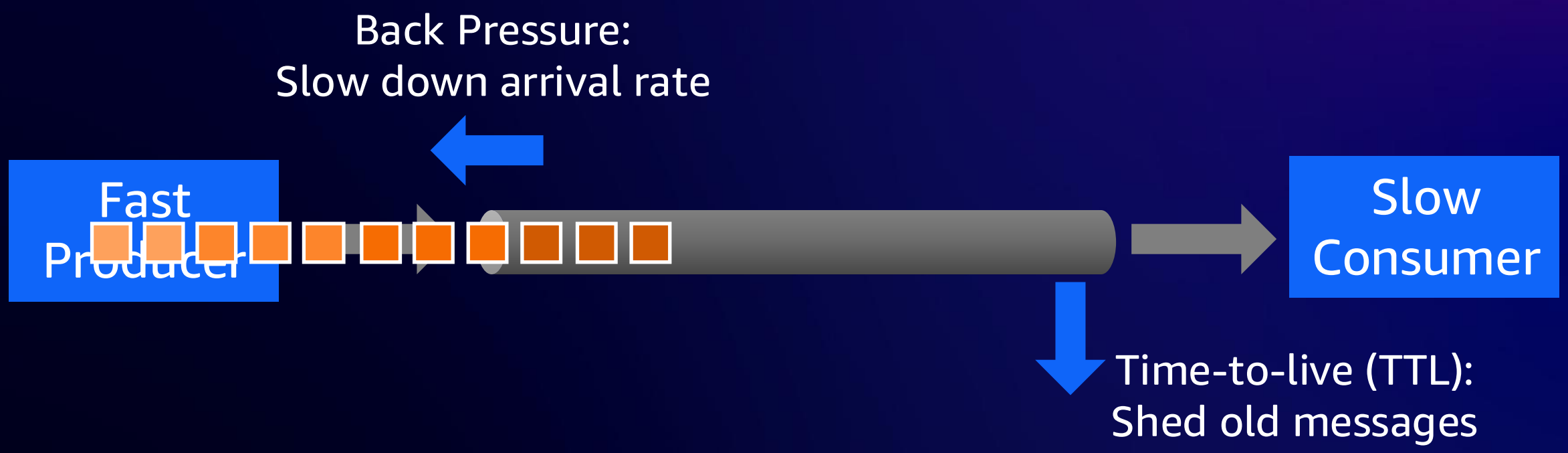
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Queues invert flow control, shape traffic



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Queues require flow control



TTL is an SQS setting, backpressure has to be built.

Even valuable messages have a meaningful time to live.

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Queues decouple control flow but require flow control.

Hard-earned experience from an integration architect

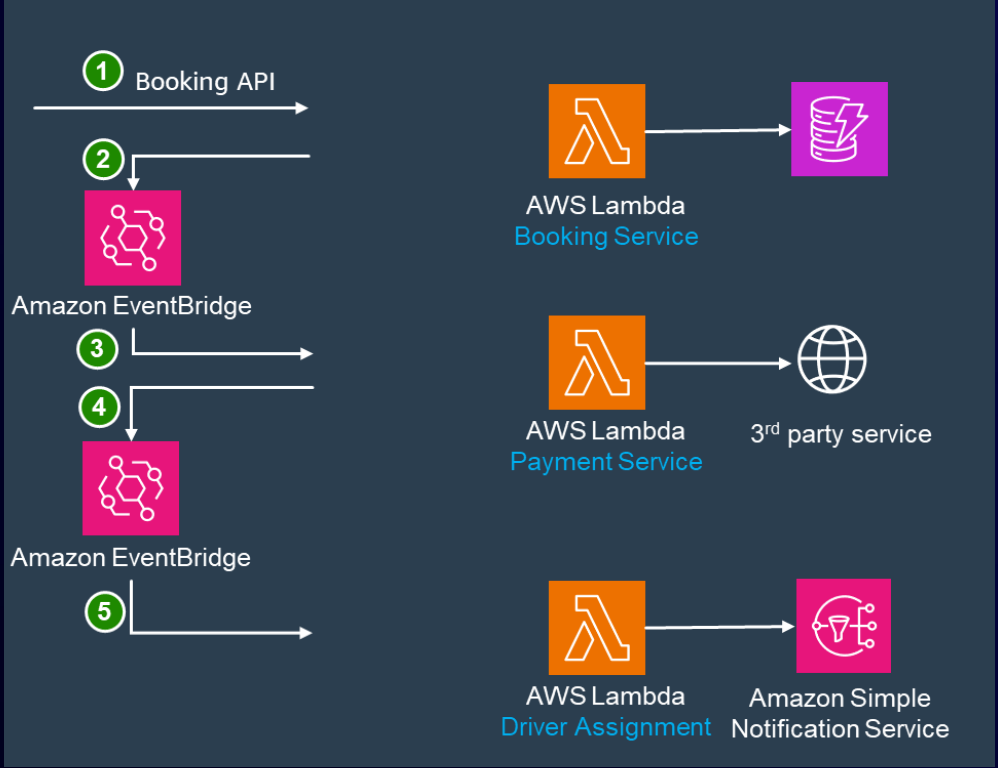
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Serverless Architectures: Coordination

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

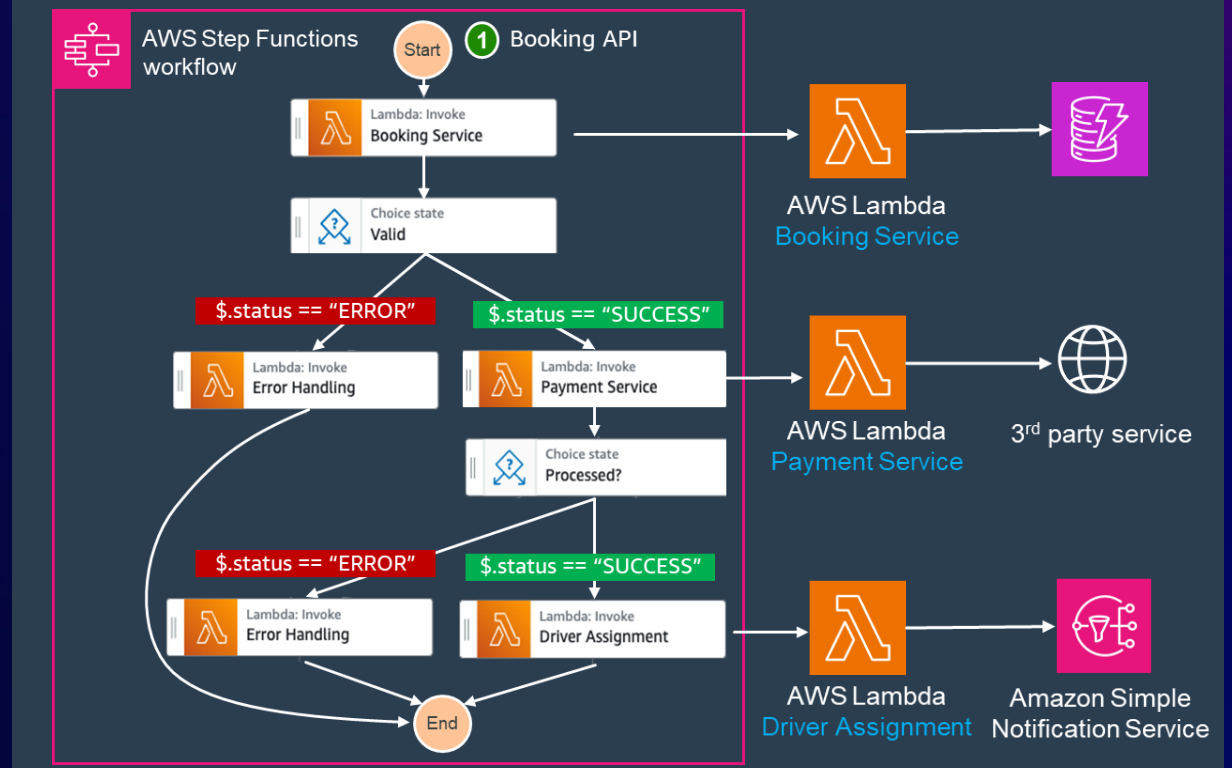
Choreography vs. Orchestration

Original Design



- ✓ Business logic can evolve easily (add, update services)
- ✓ Services can scale independently
- ➢ Lambda must publish events
- ✗ Transactional consistency across services is difficult
- ✗ End-to-end monitoring ,handling errors, and timeout is difficult

Refactored Design



- ✓ Centralized business logic in Orchestrator
- ✓ Transaction management across services
- ✓ End-to-end monitoring and error handling easier
- ✗ Tightly coupled
- ✗ Orchestrator can become single point of failure / limitation
- ✗ Single point of maintenance (can be good or bad)

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Serverless Architectures: Automation

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

“

Cloud
− Automation
= Just another data center

An opinionated architect

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Automation focused on infrastructure ("IaC")

- Object-oriented or functional
- Powerful languages
- Type systems
- Automated testing, TDD, IDE

"Build"

Development teams

- Functionality, UX
- Application architecture
- Write code

Application

"Run"

Infra / Ops teams

- Non-functional "-ilities"
- Sizing, procurement
- Click-Ops -> Write scripts

Automation

- Resource-oriented
- Prefer simple syntax (YAML)
- Strings and integers
- Text editor

https://architectelevator.com/cloud/iac-ifc-trends/

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

“

As application architecture changes, so does the role of cloud automation.

Limiting cloud automation to infrastructure is no longer adequate.

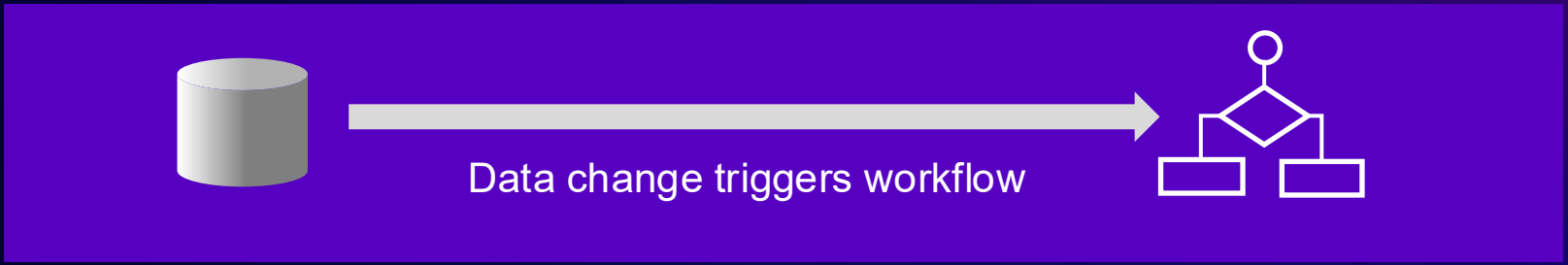
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

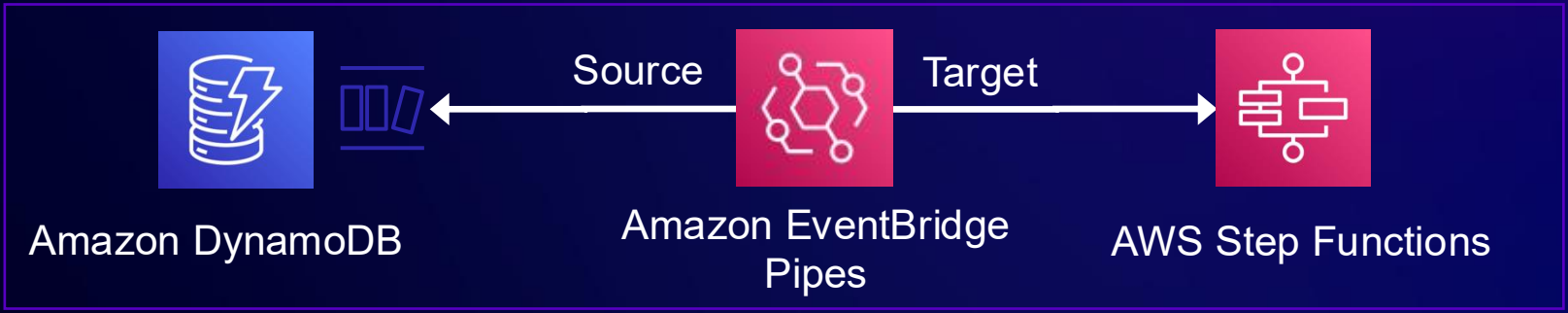
Composition with cloud automation

Architecture

Implementation (Services)

Automation Code






```
this.sourceArn = table.tableStreamArn;
this.sourceParameters = {
  dynamoDbStreamParameters: {
    startingPosition: LATEST,
    maximumBatchingWindowInSeconds: 5,
    maximumRetryAttempts: 10
  },
};
```

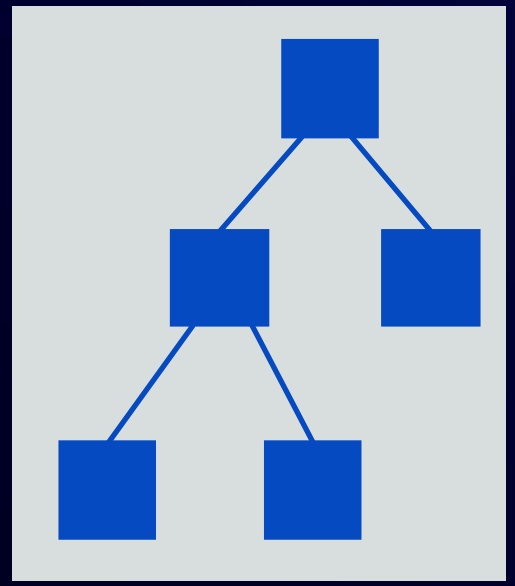
Object-Oriented (CDK)

```
Type: AWS::Pipes::Pipe
Properties:
  Description: String
  Name: String
  RoleArn: String
  Source: String
  SourceParameters:
    PipeSourceParameters
```

YAML (CloudFormation)

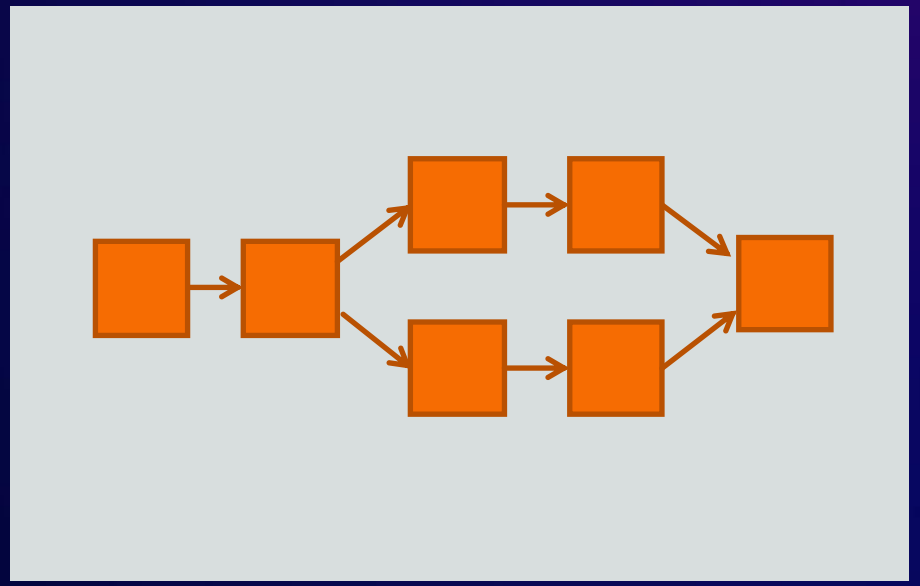
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

From IaC to AaC: Architecture as Code



Classic IaC (incl. CDK):
Resource hierarchy

Well suited for
infrastructure resources



Serverless Application Architecture:
Data and control flow

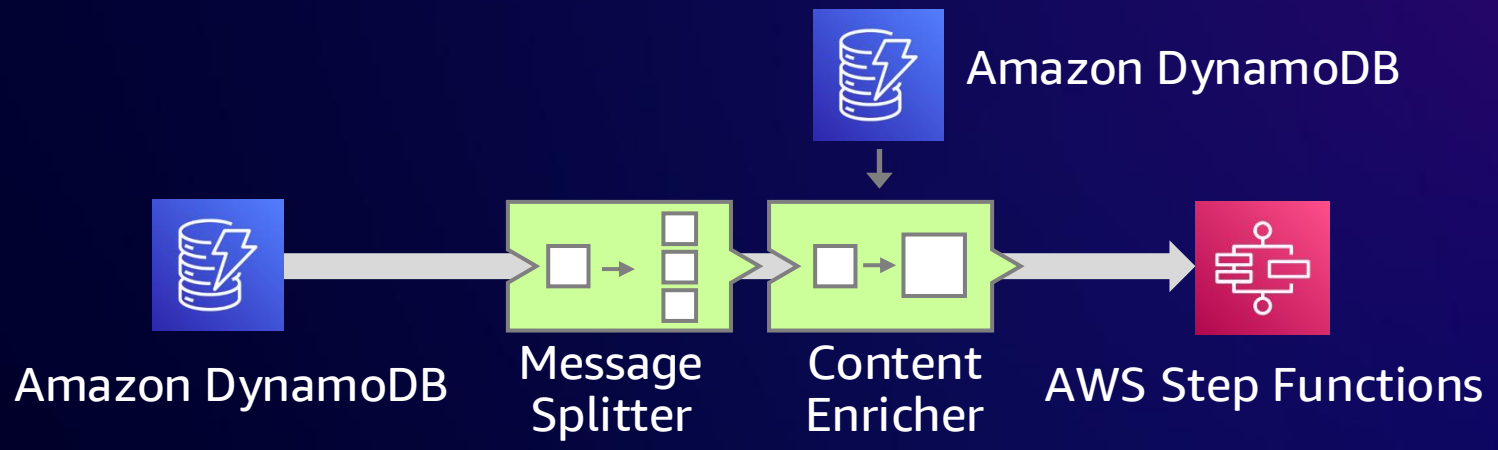
Well suited for fine-grained and
loosely coupled applications

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Source: architectelevator.com/cloud/iac-ifc-trends/

Coding in 202x: Fluent APIs, Pipes-and-Filters

Architecture



Automation Code

Implementation with CDK (Experimental)

```
const pipeline = new PipelineBuilder<IPipelineSource, Filter, Channel, IPipelineTarget>()
  .fromSource(new DynamoDBStreamSource(ordersTable))
  .with(new MessageSplitter('$.tickets', ['$.id', '$.userId'], 'common_'))
  .with(new ContentEnricher(lookupTable, { "id": "id" }, "detail"))
  .toTarget(new StepFunctionsTarget(…);

deployPipe(this, "myPipeline", pipeline, LogLevel.ALL);
```

Implementation

Icons: EnterpriseIntegrationPatterns.com

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Program to your architecture, not cloud services

Traditional Model

Architecture

Manual Translation

Implementation

Manual Translation

Automation Code

Cloud Compiler Approach

Architecture

Direct Mapping

Automation Code

Automatic Translation

Implementation

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Next-gen Serverless Automation:

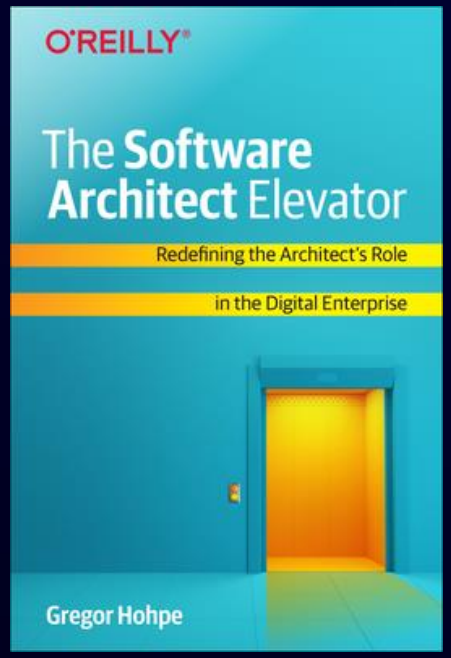
Actual

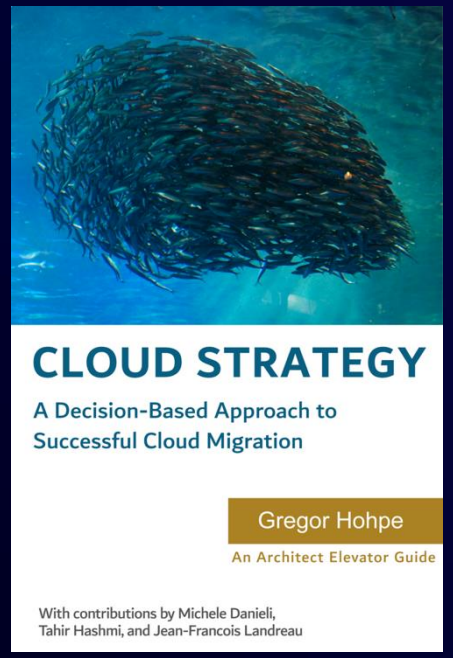
~~Infrastructure~~ **as Code**

Application Architecture

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Want more?





Gregor Hohpe
@ghohpe
www.linkedin.com/in/ghohpe

ArchitectElevator.com

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you!

Please complete the session survey in the mobile app

Gregor Hohpe

@ghohpe

Linkedin.com/in/ghohpe

ArchitectElevator.com

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.